SkillQuest Binary Tree Explorer

Application Type: Creative educational productivity application using a Binary Search Tree

Name: Joy osahita
Section: _____

Application Description

SkillQuest Binary Tree Explorer is an interactive web application that turns study goals into a visual binary tree. Each learning task, called a quest node, has a numeric score or key. The application inserts the node into the correct position using the binary-search-tree rule: smaller keys move to the left child, while larger keys move to the right child. This makes the binary tree the main data structure and the main visual element of the application.

The application is creative because it does not only display data in a list. Instead, it transforms learning tasks into a map that users can explore. Students can add new quest nodes, search for a specific key, and perform inorder, preorder, and postorder traversals. The output area shows the exact path or traversal sequence, while the UI highlights the visited nodes.

Rationale

A binary tree is useful when data needs to be organized by a key and searched efficiently. In this application, the score/key represents the task priority, difficulty, or learning order. By inserting every quest into a binary search tree, the system can demonstrate how structured data improves searching and organization. The tree visualization also helps learners understand parent nodes, left and right children, tree height, root node, and traversal order.

Main Binary Tree Operations Implemented

Operation	Purpose in the Application
Insert	Adds a new quest node and places it in the left or right subtree based on the key.
Search	Finds a quest by following the binary search path from the root.
Inorder Traversal	Displays keys in sorted ascending order.
Preorder Traversal	Visits root first, then left subtree, then right subtree.
Postorder Traversal	Visits children before the parent/root node.
Height and Count	Shows the number of nodes and the height of the current tree.

Expected Output

When opened in a browser, the application displays a dashboard-style UI with a sidebar for inputs and controls, a central binary tree canvas, and an output console. The sample tree begins with seven quest nodes. Users can insert another node, search for key 65, or run traversals. For example, inorder traversal produces: 20 -> 30 -> 40 -> 50 -> 65 -> 70 -> 85.

Screenshot of Application Output

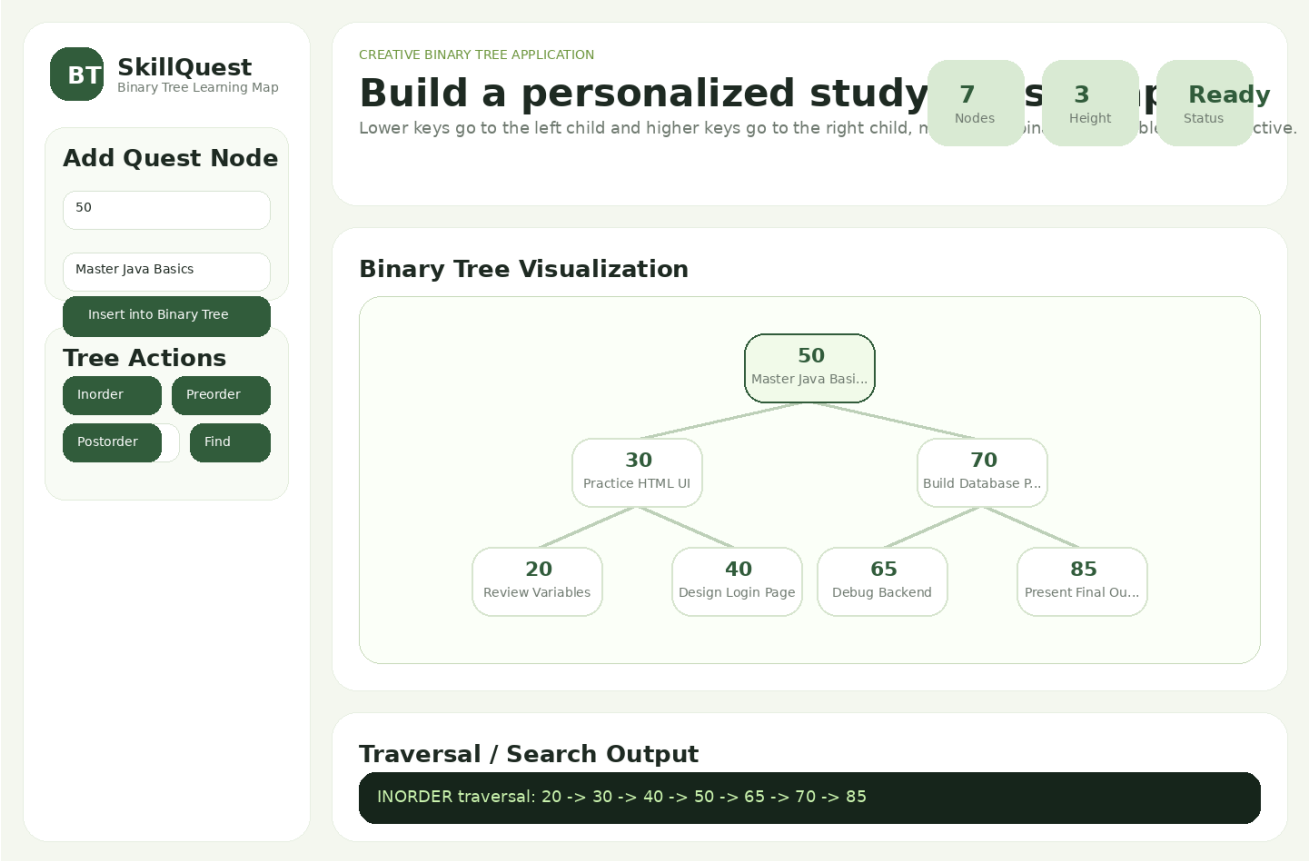

Figure 1. Main SkillQuest UI with binary tree visualization and traversal output.

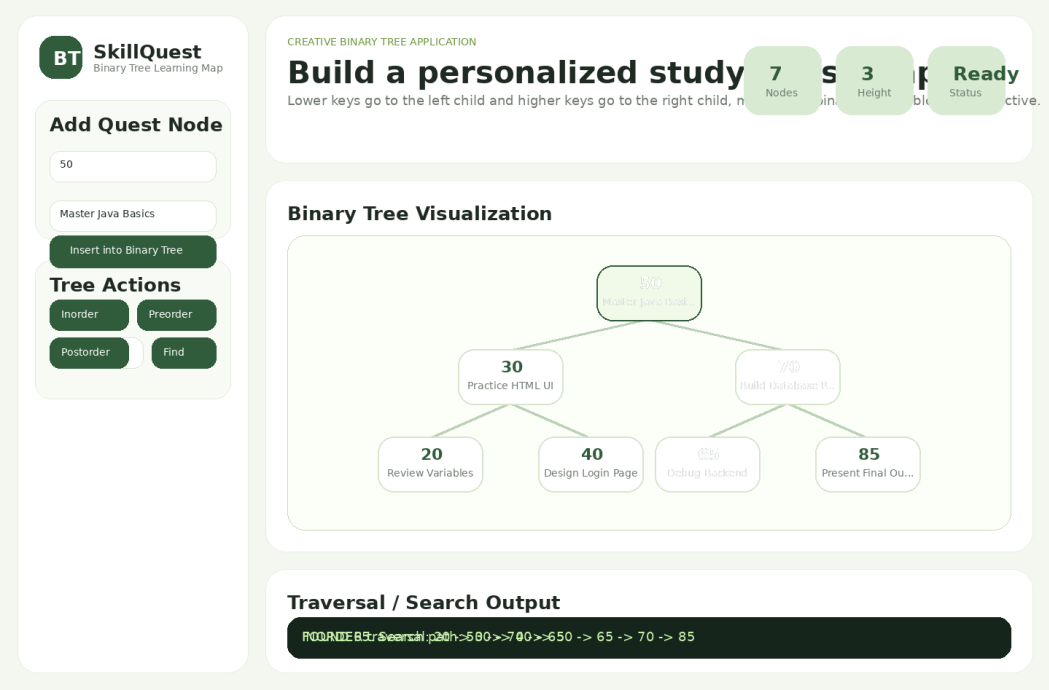

Figure 2. Search path output highlighting the visited binary tree nodes.

Screenshot of Code - HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>SkillQuest Binary Tree Explorer</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <main class="app-shell">
    <aside class="sidebar">
      <div class="brand">
        <div class="logo">BT</div>
        <div>
          <h1>SkillQuest</h1>
          <p>Binary Tree Learning Map</p>
        </div>
      </div>
      <section class="panel">
        <h2>Add Quest Node</h2>
        <label>Score / Key</label>
        <input id="keyInput" type="number" value="50" min="1" max="99" />
        <label>Quest Title</label>
        <input id="titleInput" type="text" value="Master Java Basics" />
        <button id="insertBtn">Insert into Binary Tree</button>
      </section>
      <section class="panel">
        <h2>Tree Actions</h2>
        <div class="button-grid">
          <button data-traverse="inorder">Inorder</button>
          <button data-traverse="preorder">Preorder</button>
          <button data-traverse="postorder">Postorder</button>
          <button id="resetBtn">Reset</button>
        </div>
        <label>Search Key</label>
        <div class="search-row">
          <input id="searchInput" type="number" value="65" />
          <button id="searchBtn">Find</button>
        </div>
      </section>
      <section class="legend">
        <span><i class="root-dot"></i> Root</span>
        <span><i class="left-dot"></i> Less than parent</span>
        <span><i class="right-dot"></i> Greater than parent</span>
      </section>
    </aside>

    <section class="content">
      <header class="hero-card">
        <div>
          <p class="eyebrow">Creative Binary Tree Application</p>
          <h2>Build a personalized study quest map where every node is organized by score.</h2>
          <p>Lower keys go to the left child and higher keys go to the right child, making the binary tree visible and interactive.</p>
        </div>
        <div class="stats">
          <div><strong id="nodeCount">0</strong><span>Nodes</span></div>
          <div><strong id="heightCount">0</strong><span>Height</span></div>
          <div><strong id="lastAction">Ready</strong><span>Status</span></div>
        </div>
      </header>

      <section class="tree-card">
        <div class="card-title">
          <h2>Binary Tree Visualization</h2>
          <p>Click traversal buttons to highlight how the tree is visited.</p>
        </div>
        <div id="treeCanvas" class="tree-canvas"></div>
      </section>

      <section class="output-card">
        <h2>Traversal / Search Output</h2>
        <div id="output" class="output">Insert or traverse nodes to see output here.</div>
      </section>
    </section>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

Screenshot of Code - CSS

```css
:root {
  --bg: #f4f7ef;
  --panel: #ffffff;
  --green: #315c3b;
  --mint: #d9ead3;
  --lime: #a8c745;
  --ink: #1e2a22;
  --muted: #6f7a70;
  --line: #bdd0b6;
  --shadow: 0 20px 45px rgba(35, 70, 42, 0.14);
}
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Inter, Arial, sans-serif;
  color: var(--ink);
  background: radial-gradient(circle at top left, #e3f4d2, var(--bg) 40%, #edf2e9);
}
.app-shell { min-height: 100vh; display: grid; grid-template-columns: 340px 1fr; gap: 24px; padding: 24px; }
.sidebar, .hero-card, .tree-card, .output-card { background: rgba(255,255,255,.86); border: 1px solid rgba(255,255,255,.9); border-radius: 28px; box-shadow: var(--shadow); }
.sidebar { padding: 24px; backdrop-filter: blur(17px); }
.brand { display: flex; align-items: center; gap: 14px; margin-bottom: 28px; }
.logo { width: 50px; height: 50px; border-radius: 20px; display: grid; place-items: center; background: linear-gradient(135deg, var(--green), var(--lime)); color: white; font-weight: 900; letter-spacing: -1px; }
h1, h2, p { margin: 0; }
.brand h1 { font-size: 26px; }
.brand p, .card-title p, .hero-card p, .legend, label { color: var(--muted); }
.panel { padding: 18px; border-radius: 22px; background: #f8fbf5; margin-bottom: 18px; border: 1px solid #e3eddc; }
.panel h2, .output-card h2, .tree-card h2 { font-size: 18px; margin-bottom: 14px; }
label { display: block; font-size: 13px; font-weight: 700; margin: 12px 0 8px; }
input { width: 100%; border: 1px solid #d5e4d1; border-radius: 14px; padding: 13px 14px; font-size: 15px; outline: none; background: white; }
button { border: none; cursor: pointer; border-radius: 14px; padding: 13px 14px; font-weight: 800; color: white; background: var(--green); transition: transform .2s ease, box-shadow .2s ease, background .2s ease; }
button:hover { transform: translateY(-2px); box-shadow: 0 12px 22px rgba(49, 92, 59, .24); background: #2448d2d; }
#insertBtn { width: 100%; margin-top: 14px; background: linear-gradient(135deg, var(--green), #71f173); }
.button-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }
.search-row { display: grid; grid-template-columns: 1fr 86px; gap: 10px; }
.legend { display: grid; gap: 10px; font-size: 14px; padding-left: 8px; }
.legend i { width: 12px; height: 12px; border-radius: 50%; display: inline-block; margin-right: 8px; }
.root-dot { background: var(--green); }.left-dot { background: #72a85b; }.right-dot { background: var(--lime); }
.content { display: grid; gap: 24px; }
.hero-card { display: flex; justify-content: space-between; gap: 24px; align-items: center; padding: 30px; }
.eyebrow { text-transform: uppercase; font-size: 12px; font-weight: 900; letter-spacing: 2px; color: #6d963d !important; margin-bottom: 8px; }
.hero-card h2 { max-width: 760px; font-size: clamp(28px, 3vw, 48px); line-height: 1.02; margin-bottom: 12px; }
.stats { display: grid; grid-template-columns: repeat(3, 120px); gap: 12px; }
.stats div { background: var(--mint); border-radius: 22px; padding: 16px; text-align: center; }
.stats strong { display: block; font-size: 26px; color: var(--green); }
.stats span { color: var(--muted); font-size: 12px; font-weight: 800; }
.tree-card, .output-card { padding: 24px; }
.card-title { display: flex; justify-content: space-between; align-items: end; gap: 16px; margin-bottom: 22px; }
.tree-canvas { min-height: 470px; position: relative; overflow: auto; border-radius: 24px; background: linear-gradient(180deg, #fbfff8, #edf6e8); border: 1px dashed #c8dab; }
.edge { position: absolute; height: 3px; background: var(--lime); transform-origin: left center; border-radius: 6px; }
.node { position: absolute; width: 145px; min-height: 76px; transform: translate(-50%, -50%); border-radius: 22px; background: white; border: 2px solid #d7e5cf; display: grid; place-items: center; text-align: center; padding: 10px; box-shadow: 0 10px 26px rgba(30, 42, 34, .12); transition: .25s ease; }
.node b { display: block; font-size: 21px; color: var(--green); }
.node small { display: block; color: var(--muted); font-weight: 700; margin-top: 3px; }
.node.root { border-color: var(--green); background: #f1fae8; }
.node.highlight { background: var(--lime); color: white; border-color: #7ca02f; transform: translate(-50%, -50%) scale(1.08); }
.node.highlight b, .node.highlight small { color: white; }
.output { min-height: 72px; border-radius: 18px; padding: 18px; background: #16251b; color: #c9f2ad; font-family: Consolas, monospace; line-height:
```

Screenshot of Code - JavaScript Binary Tree Logic

```
class TreeNode {
  constructor(key, title) {
    this.key = key;
    this.title = title;
    this.left = null;
    this.right = null;
  }
}

class BinarySearchTree {
  constructor() { this.root = null; }

  insert(key, title) {
    const newNode = new TreeNode(key, title);
    if (!this.root) { this.root = newNode; return 'Inserted as root node'; }
    let current = this.root;
    while (true) {
      if (key === current.key) { current.title = title; return 'Updated existing node'; }
      if (key < current.key) {
        if (!current.left) { current.left = newNode; return `Inserted ${key} to the LEFT of ${current.key}`; }
        current = current.left;
      } else {
        if (!current.right) { current.right = newNode; return `Inserted ${key} to the RIGHT of ${current.key}`; }
        current = current.right;
      }
    }
  }

  search(key) {
    const path = [];
    let current = this.root;
    while (current) {
      path.push(current.key);
      if (key === current.key) return { found: true, path, node: current };
      current = key < current.key ? current.left : current.right;
    }
    return { found: false, path, node: null };
  }

  traverse(type) {
    const result = [];
    const visit = node => { if (node) result.push(node.key); };
    const inorder = node => { if (!node) return; inorder(node.left); visit(node); inorder(node.right); };
    const preorder = node => { if (!node) return; visit(node); preorder(node.left); preorder(node.right); };
    const postorder = node => { if (!node) return; postorder(node.left); postorder(node.right); visit(node); };
    ({ inorder, preorder, postorder }[type])(this.root);
    return result;
  }

  count(node = this.root) { return node ? 1 + this.count(node.left) + this.count(node.right) : 0; }
  height(node = this.root) { return node ? 1 + Math.max(this.height(node.left), this.height(node.right)) : 0; }
}

const tree = new BinarySearchTree();
const defaults = [
  [50, 'Master Java Basics'], [30, 'Practice HTML UI'], [70, 'Build Database Plan'],
  [20, 'Review Variables'], [40, 'Design Login Page'], [65, 'Debug Backend'], [85, 'Present Final Output']
];
const canvas = document.getElementById('treeCanvas');
const output = document.getElementById('output');
const nodeCount = document.getElementById('nodeCount');
const heightCount = document.getElementById('heightCount');
const lastAction = document.getElementById('lastAction');

defaults.forEach(([key, title]) => tree.insert(key, title));
renderTree();

function renderTree(highlights = []) {
  canvas.innerHTML = '';
  const positions = [];
  const levelGap = 105;
  const minXGap = 86;
  let order = 0;

  function assign(node, depth) {
    if (!node) return;
    assign(node.left, depth + 1);
    positions.push({ node, x: 95 + order * minXGap, y: 70 + depth * levelGap, depth });
    order++;
    assign(node.right, depth + 1);
  }
  assign(tree.root, 0);
  const maxX = Math.max(760, 190 + positions.length * minXGap);
  canvas.style.minWidth = `${maxX}px`;

  function posOf(node) { return positions.find(p => p.node === node); }
  positions.forEach(({ node }) => {
    if (node.left) drawEdge(posOf(node), posOf(node.left));
    if (node.right) drawEdge(posOf(node), posOf(node.right));
  });
  positions.forEach(({ node, x, y, depth }) => {
    const div = document.createElement('div');
    div.className = `node ${depth === 0 ? 'root' : ''} ${highlights.includes(node.key) ? 'highlight' : ''}`;
    div.style.left = `${x}px`; div.style.top = `${y}px`;
    div.innerHTML = `<b>${node.key}</b><small>${node.title}</small>`;
    canvas.appendChild(div);
  });
  nodeCount.textContent = tree.count();
  heightCount.textContent = tree.height();
}

function drawEdge(a, b) {
  const dx = b.x - a.x, dy = b.y - a.y;
  const length = Math.sqrt(dx * dx + dy * dy);
  const edge = document.createElement('div');
  edge.className = 'edge';
  edge.style.width = `${length}px`;
  edge.style.left = `${a.x}px`; edge.style.top = `${a.y + 40}px`;
  edge.style.transform = `rotate(${Math.atan2(dy, dx)}rad)`;
  canvas.appendChild(edge);
}

document.getElementById('insertBtn').addEventListener('click', () => {
  const key = Number(document.getElementById('keyInput').value);
  const title = document.getElementById('titleInput').value.trim() || 'Untitled Quest';
  const message = tree.insert(key, title);
  output.textContent = message;
  lastAction.textContent = 'Inserted';
  renderTree([key]);
});

document.querySelectorAll('[data-traverse]').forEach(btn =>
```

Files Included

The project folder includes index.html, styles.css, script.js, a README file, screenshot images, and this PDF report. The application can be run by opening index.html in a browser.